

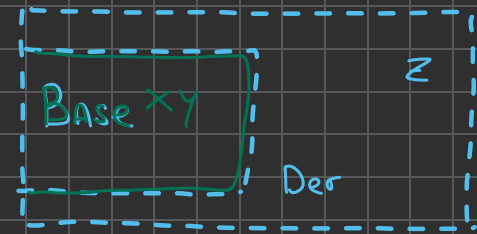
Наследяване

Наследяване - метод за създаване на нов клас от вече съществуващ такъв, който да разширява неговите данни и поведение

```
class Base {  
    public:  
        int x;  
        f();  
    private:  
        int y;  
        g();  
}
```

```
class Der : public Base {
```

```
    public:  
        int z;
```



Inheritance vs Composition

Наследяване	Композиция
is-a	has-a
функционалността, чрез която един обект придобива характеристиките на един или повече други обекти	Използването на обект вътре в друг обект
дефиниран по време на компилиране - Compile time	дефиниран по време на изпълнение - Runtime
"разкрива" public & protected данни на базовия клас	вътрешните данни не са разкрити/достъпни, а взаимодействат чрез публичен интерфейс (public interface)
Access control/Visibility	Access control/Visibility

Пример 3А
госпожа

```
class Base {
    int y;
public:
    int x;
    f();
};
```

```
class Der: public Base {
```

```
    g() {
        f();
        y++; }
    int x;
```

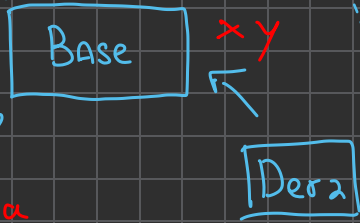
→ нямаме gomen
→ мога X shadow-ва
мога на Base

sizeof(Der) = der + base

Base class member access specifier	Type of Inheritance		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

Наследниците могат да бъдат позовавани като параметри на ф-ии, които приемат обекти от базовия клас.

class - no default name space
struct - no default name space
private
public



Der1 obj;
Base * ptr = &obj;
Base &ref = obj;

ptr -> x++;
ptr -> y++;
~~ptr -> a++;~~ (майка не е в мейна аemento)

Пример геме в мейна

```
f(const Base * ptr) {
    ptr -> x;
    ptr -> y;
}
```

✓ Base o1; f(801);
✓ Der1 o2; f(802);
✓ Der2 o3; f(803);

Пример майка не влиза в детето

$f(\text{Der} * pto)$

Der obj;
Base * pto = &obj;

~~$f(pto)$~~ → майката не влиза в детето си

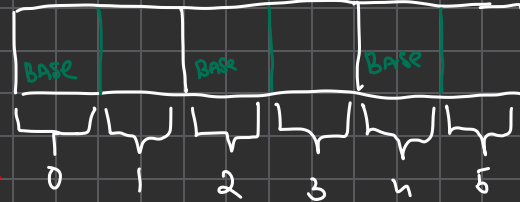
Пример с масив

Der arr[3];

$f(\text{const Base} * arr, \text{size})$

~~$f(arr, 3)$~~

→ пошото е, че се компилира!



инкрементира с размера на Base

Конструктори при наследяване:

→ Конструкторът на наследника трябва да каже кой кон-р на базовия да се викне.

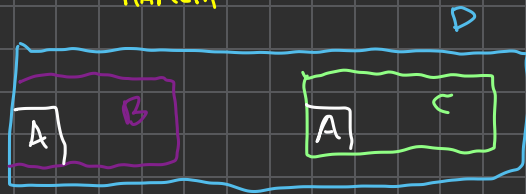
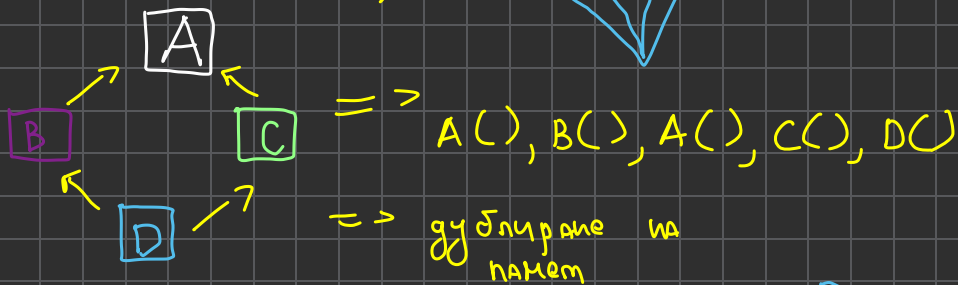
! Ако не каже
↓
се вика дефолтният

! Ако няма дефолтен
↓
компилационна грешка

Base се крие за себе си (поjavява в copy cons. Base(other), тог си измисля новото)

```
1 class Base {
2 public:
3     int x;
4 };
5
6 class Derived : private Base { };
7
8 void f(Base& b) { }
9
10 int main() {
11     Derived d;
12     f(d); // Compilation error
13     // Conversion from Derived to Base is inaccessible
14     // because of private inheritance
15 }
```

Динамичен проблем:



=> Виртуално наследяване

X : virtual Y

=> не more

Всички наследник на X (прехи и прехи) се извикват да се изпълнят на Y. X създава Y.

B(), C(), A(), D()
 L -> имаме 1 A
 споделено за B и C

A: [A]

B: [x B] [A]

$$\text{sizeof}(B) \geq \text{sizeof}(B) + \text{sizeof}(A)$$

C: [x C] [A]

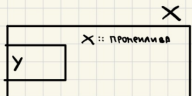
D: [x B] [x C] [A]

$$\text{sizeof}(D) \geq \text{sizeof}(B) + \text{sizeof}(C) + \text{sizeof}(A)$$

Стандартно наследяване VS

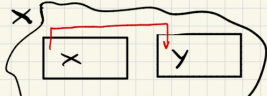
Виртуално наследяване

class X : public Y { ... }



// Адресът на Y и X е еднакъв

class X: virtual public Y { ... }



// Частта от Y не е физически съдържа в инстанцията на X.

Адресите на X и Y не са еднакви и са различни.

{ X obj;
 Y* basePtr = obj;
}



ВЪРХОВНО НАСЛЕДЯВАНЕ: class X: virtual public Y { }

Така всеки наследник на X е длъжен да каже експлицитно как да се създаде Y.

- Прехвърля се отговорността за създаването на Y "надолу".

C: A(...) { }

E C(): B(), A(...) { }

Ако A няма дефолтен и това не сме го записали, то тогава да се компилира.

Понякога наследяването е виртуално C трябва да каже как да се създаде A.

Ако това сме го изпуснали и A има дефолтен, то ще се извика тогава.